

20-SLIDE PRESENTATION: MICROSERVICES ARCHITECTURE, DECOMPOSITION, SECURITY & PLATFORM DEPLOYMENT

Course: Cloud Computing & Security

Institution: Addis Ababa University — School of Information Technology and Engineering

Instructor: Daniel

Document: 20-Slide Presentation Deck

Repository: <https://github.com/Abyma2/Easyservice>

Group Members (Group Two)

#	First Name	Last Name
1	**Abel**	**Seleshe**
2	**Baheran**	**Tesfaye**
3	**Nebiyu**	**Yohannes**
4	**Wondesen**	**Teshale**
5	**Esrom**	**Basazinaw**
6	**Yohannes**	**Seyum**

SLIDE 1: Title & Executive Summary

- **Title:** End-to-End Microservices Platform Architecture, Security & Production Operations
- **System:** EasyService Marketplace Platform
- **Key Focus:** DDD Decomposition, Distributed Transactions (Saga), Docker & Kubernetes, OAuth2/JWT Security, Istio Service Mesh, and Full Pyramid Testing.

SLIDE 2: Chapter 1 — Monolith vs. Microservices Architecture

- **Monolith:** Single deployment unit, shared DB, single point of failure.
- **Microservices:** Autonomous, independently deployable services organized around business bounded contexts.
- **Newman's Core Principle:** Model around business domains; hide implementation details.

SLIDE 3: Practical Lab 1 — Building & Containerizing First Microservice

- Built Spring Boot REST Service with Docker multi-stage containerization.

- **Key Command:** `docker build -t easyservice/user-service:v1.0 .`
 - Executed local API verification via cURL / Postman.
-

SLIDE 4: Chapter 2 — Microservice Decomposition & Domain-Driven Design (DDD)

- Identified 3 core Bounded Contexts: **User Identity Context**, **Marketplace Context**, **Booking Context**.
 - Applied **Database per Service Pattern**: Enforced data encapsulation with zero shared DB tables across services.
-

SLIDE 5: Practical Lab 2 — Monolith Decomposition Engine

- Monolith broken into 4 core services: **UserService**, **ListingService**, **BookingService**, **PaymentService**.
 - Defined clear OpenAPI REST contracts for cross-context communication.
-

SLIDE 6: Chapter 3 — Communication & Data Management

- **Synchronous**: REST & gRPC for latency-critical query APIs.
 - **Asynchronous**: Event-driven architecture via Kafka / RabbitMQ for event processing.
 - **Event Sourcing**: Storing state change logs as immutable domain event streams.
-

SLIDE 7: Chapter 3 — Distributed Transactions & The Saga Pattern

- Solved dual-write distributed transaction problems using the **Choreography Saga Pattern**.
 - **Success Flow**: **BookingCreated** $\rightarrow$ **PaymentProcessed** $\rightarrow$ **BookingConfirmed**.
 - **Compensating Action**: If payment fails, trigger **CancelBooking** & restore inventory stock.
-

SLIDE 8: Practical Lab 3 — Event Messaging & Saga Execution

- Configured RabbitMQ exchanges and queues.
 - Implemented **@KafkaListener** / **@RabbitListener** event handlers with retry mechanisms and dead-letter queues.
-

SLIDE 9: Chapter 4 — Microservices Development & Container Orchestration

- Package microservices into immutable Docker container images.

- Managed multi-container local environments using `docker-compose.yml`.
-

SLIDE 10: Chapter 4 — Kubernetes Cluster Deployment

- Orchestrated pods using Kubernetes **Deployments**, **Services**, and **Ingress Controllers**.
 - Configured declarative resource limits (`cpu: 250m`, `memory: 256Mi`) and horizontal pod autoscaling.
-

SLIDE 11: Practical Lab 4 — Kubernetes Cluster & API Gateway Deployment

- Created Kubernetes manifests for microservices behind NGINX API Gateway.
 - Managed application settings using `ConfigMap` and sensitive database secrets using Kubernetes `Secrets`.
-

SLIDE 12: Chapter 5 — Microservices Security & Zero Trust Architecture

- Enforced stateless **OAuth 2.0** & **JWT Bearer Token** authentication.
 - API Gateway verifies JWT signature and propagates user context claims via internal HTTP headers.
-

SLIDE 13: Chapter 5 — Reliability: Circuit Breakers & Resilience4j

- Prevent cascading service failures using **Resilience4j Circuit Breakers**.
 - States: `CLOSED` (Normal) $\rightarrow$ `OPEN` (Failures detected, fallback executed) $\rightarrow$ `HALF-OPEN` (Probe recovery).
-

SLIDE 14: Chapter 5 — Observability: OpenTelemetry, Prometheus & Grafana

- Distributed tracing with OpenTelemetry propagates `traceId` across services.
 - Prometheus scrapes microservice metrics; Grafana visualizes real-time performance and error rates.
-

SLIDE 15: Practical Lab 5 — Security Hardening & Observability Integration

- Implemented role-based authorization guards on endpoints.
- Deployed Prometheus + Grafana monitoring dashboard with real-time request latency alerts.

SLIDE 16: Chapter 6 — Operations & Service Mesh (Istio & Envoy)

- Istio sidecar proxies intercept inter-service traffic automatically.
 - Enforced **Mutual TLS (mTLS)** for automatic zero-trust wire encryption between services.
-

SLIDE 17: Chapter 6 — Traffic Management & Canary Deployments

- Istio VirtualServices route traffic dynamically (e.g. 90% traffic to **v1.0**, 10% traffic to **v2.0** canary release).
 - Enforced rate limiting and automated retry policies at the network layer.
-

SLIDE 18: Chapter 6 — Testing Strategies & The Test Pyramid

- Multi-tier automated testing:
 - **Unit Tier**: JUnit 5 + Mockito testing domain rules.
 - **Integration Tier**: Spring Boot Test + MockMvc validating REST APIs.
 - **System E2E Tier**: Selenium WebDriver using **Page Object Model (POM)** pattern.
-

SLIDE 19: Practical Lab 6 — Production Platform & CI/CD Deployment

- Deployed full production-ready microservices application to Kubernetes with Istio mTLS enabled.
 - Automated CI/CD pipeline in GitHub Actions / Jenkins with automated regression test validation.
-

SLIDE 20: Conclusions & Key Takeaways

- Microservices enable rapid, autonomous delivery when combined with domain-driven boundaries, automated testing, container orchestration, and continuous integration.
- The EasyService reference architecture proves compliance with enterprise microservice patterns and cloud security standards.